ISTANBUL TECHNICAL UNIVERSITY

COMPUTER ENGINEERING DEPARTMENT

BLG 212E

MICROPROCESSOR SYSTEMS
Homework 2

150220939 : Madina & Alzhanova

SPRING 2024

Contents

1 INTRODUCTION **1**

2 MATERIALS AND METHODS **1**

2.1 Part 1 1

2.2 Part 2 - 3 2

2.3 Part 4 3

3 RESULTS **4**

4 CONCLUSION **5**

1 INTRODUCTION

In this assignment, I have implemented timer interrupt by using SysTick. When the code runs, it fills memory with different numbers (representing colors) drawing some figures and letters according to commands in predefined array.

2 MATERIALS AND METHODS

2.1 Part 1

SysTick registers are defined in the very beginning of the code according to their addresses from ARM datasheet shown in Figure 1.

Table 11-4 SysTick Timer register summary

Address offset	Name	Type	Reset value	Description
0xE000E010	SYST_CSR	RW	0x00000000	SysTick Control and Status Register
0xE000E014	SYST_RVR	RW	0x00000000	SysTick Reload Value Register
0xE000E018	SYST_CVR	RW	0x00000000	SysTick Current Value Register
0xE000E01C	SYST_CALIB	RO	0x00000000	SysTick Calibration Value Register

Figure 1: Systick Registers from Datasheet

System clock is 10MHz. The value for STRELOAD register is calculated as following:

$$\begin{aligned}\text{Reload Value} &= (\text{Target Delay} \times \text{Clock Frequency}) - 1 \\ &= (1 \text{ sec} \times 10,000,000 \text{ Hz}) - 1 \\ &= 10,000,000 - 1 \\ &= 9,999,999_{10} \\ &= \mathbf{0x0098967F}\end{aligned}$$

Then, RELOAD_VAL = 0x0098967F. The values and addresses of registers are defined as in Figure 2.

```
STCTRL_ADDR     EQU 0xE000E010      ; SysTick Control and Status Register from datasheet
STRELOAD_ADDR   EQU 0xE000E014      ; SysTick Reload Value Register
STCURR_ADDR     EQU 0xE000E018      ; SysTick Current Value Register
RELOAD_VAL      EQU 0x0098967F      ; 10,000,000 - 1 = 9 999 999 -> 0x0098967F
```

Figure 2: Register contents

I put a breakpoint inside Handler function(Figure 3) to test if the delay is indeed 1 second. However, i get 0.833 seconds delay for calculated reload value. This is because for default settings, the clock frequency is not 10MHz, but 12MHz. But i left the reload as it is, and interrupt is called every 0.833 seconds as in Figure 4.

```
178 ; override handler:
179 SysTick_Handler FUNCTION
180         EXPORT  SysTick_Handler     ; Export handler to override
181         PUSH    {R0, R1, LR}        ; save registers
182         LDR     R0, =varb ; get varb
183         MOVS     R1, #1  ; R1 <- 1
184         STR     R1, [R0]            ; Set varb
185         POP     {R0, R1, PC}        ; restore
186         ENDFUNC
```

Figure 3: Testing delay

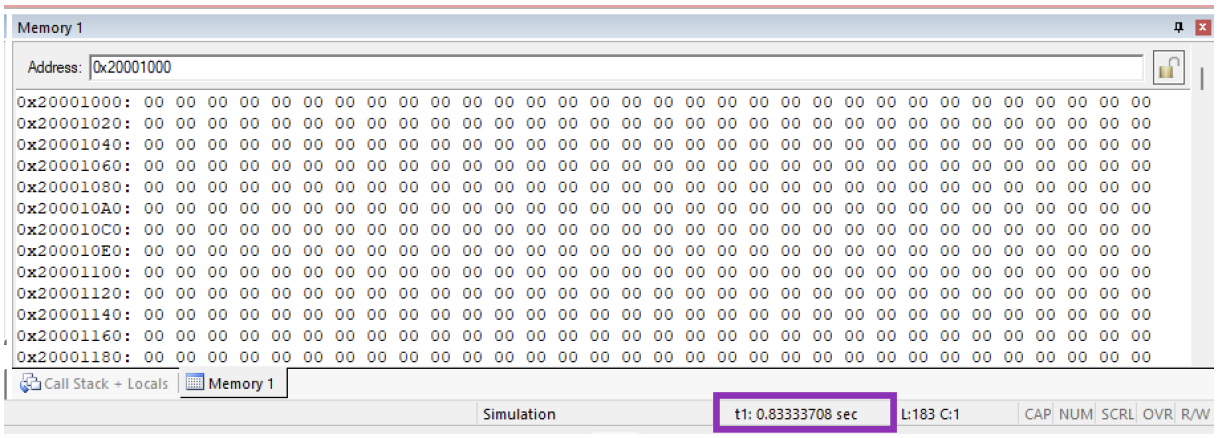

Figure 4: Unit test delay

2.2 Part 2 - 3

In handler, i added accessing the array and fetching the current instruction (using vari). Also handler sets varb flag showing that new value is ready. Going to next index is done inside main code, because ISR should be as short as possible.

Inside main, there is process_input function, which gets instruction from ISR. Increments index (vari) to point to the next instruction. It checks if the current instruction is end or the array. Then decodes by separating MSB and LSB, according to which **set_color** or **start_move** is called. **set_color** updates varc variable, and in **start_move**, **move_step** repeatedly calls **update_pos_paint** function until length becomes zero. **update_pos_paint** increments or decrements the coordinates according to the direction. **update_pos_paint** has do_right, do_left, do_up, do_down, check_x and check_y functions inside where the boundaries of grid are checked. To match the grid shown in task, I use

32x8 grid. Finally, for painting, value at varc is written to the memory at coordinates calculated from offset, x and y coordinates.

2.3 Part 4

In this implementation, the design is constrained to use only two registers: R0 and R1. To maintain data integrity between the main loop and the interrupt service routine (ISR), stack operations are essential.
Inside ISR, all registers in use are stores into stack: PUSH R0, R1, LR. When ISR finishes, all of them are restored from the stack, so ISR doesn't change normal flow of main code and restores register values: POP R0, R1, PC.
Also inside process_input, i need more registers, so in order to increment index of array, i push R1 into stack, do index increment, then restore R1 back by POP R1.
The following table describes the purpose and register usage of the key subroutines implemented in the assembly program.

Function Name	**Registers Used**	**Description**
`__main`	R0, R1	The entry point of the program. Initializes the SysTick timer registers (CTRL, RELOAD, CURRENT) and starts the main infinite loop for polling the interrupt flag.
`SysTick_Handler`	R0, R1, LR	The interrupt service routine triggered every 1 second. It fetches the current instruction from the array using the `vari` index, stores it in the shared variable `var_inst`, and sets the `varb` flag to notify the main loop.
`process_input`	R0, R1	Checks the valid input range, increments the array index (`vari`), and decodes the instruction byte. It branches to either `set_color` or `start_move` based on the MSB (command) of the instruction.
`update_pos_paint`	R0, R1, LR	Updates the cursor's X and Y coordinates based on the direction in R0. It checks boundary conditions (0-31 for X, 0-7 for Y) and writes the current color (`varc`) to the calculated memory address.
`start_move`	R0, R1	A helper loop that executes the movement logic. It calls `update_pos_paint` repeatedly for the number of steps specified in the instruction length (LSB).

Table 1: Documentation of Functions

3 RESULTS

Shorter example draws a figure like loop with one color, while longer example writes "ITUCS" with different colors for each letter.

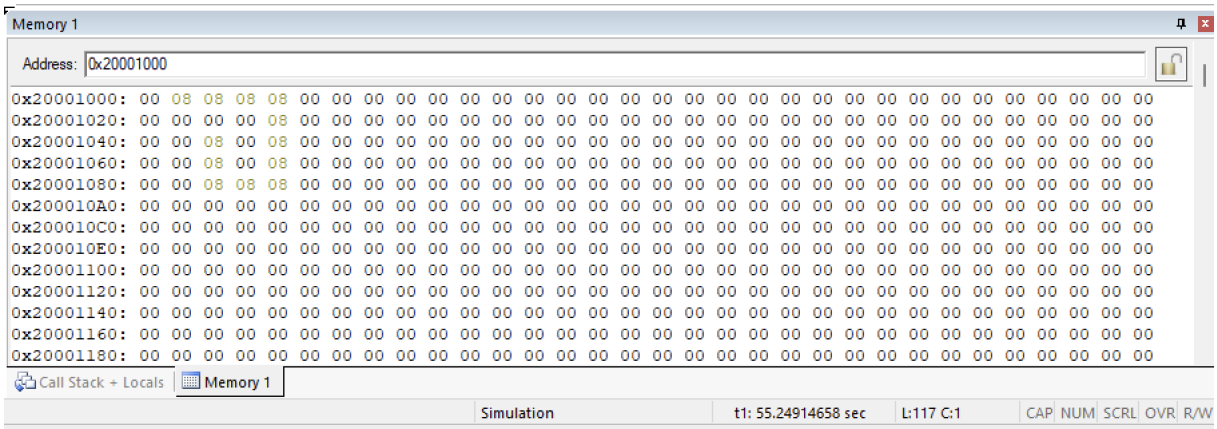

Figure 5: Result of shorter example

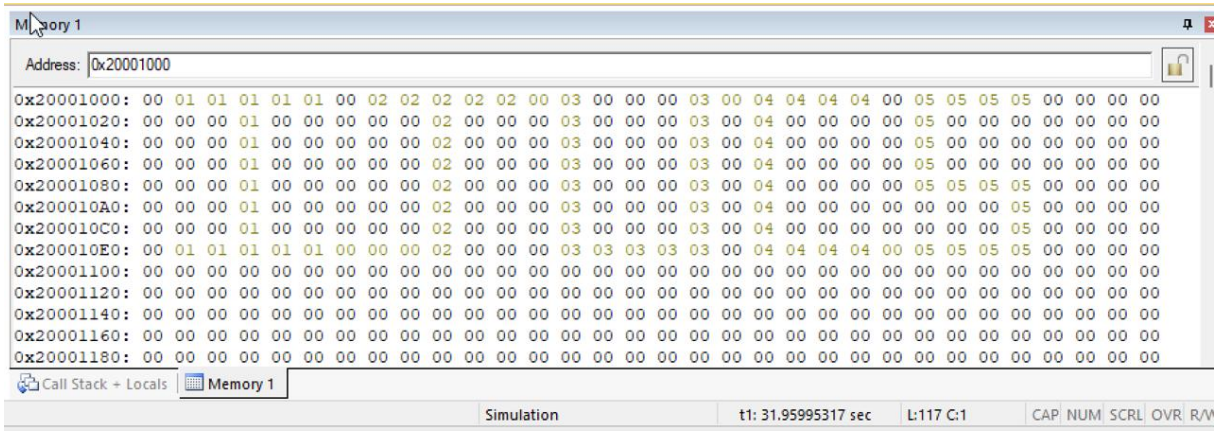

Figure 6: Result of longer example

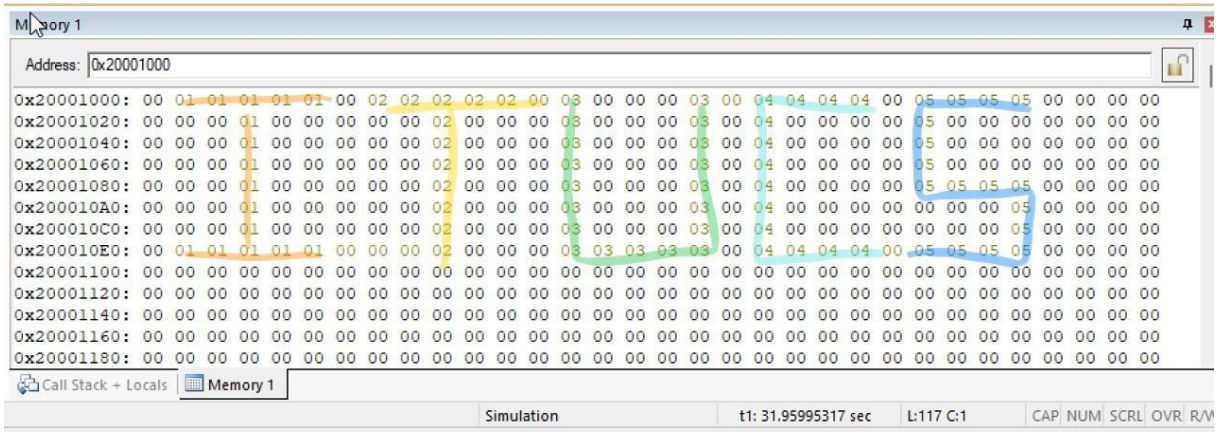

Figure 7: Longer example: ITUCS

4 CONCLUSION

This project successfully implemented a timer-driven drawing application on the ARM Cortex-M0 architecture. The assignment demonstrated the critical role of interrupts in handling asynchronous events and the importance of efficient register management.
The discrepancy observed between the theoretical 10 MHz clock and the simulator's 12 MHz clock highlighted the need for hardware-aware calibration in embedded systems. Furthermore, the constraint of using only two registers necessitated careful stack usage to preserve context between the main loop and the ISR. The final output visuals confirm that the decoding logic, memory addressing, and interrupt handling were implemented correctly.